

Producer-Consumer System Analysis

Maria Charisi

May 2025
Github Repo

Abstract

This assignment evaluates the behavior of a multi-threaded system where p producer threads enqueue function pointers of type `struct workFunction` into a shared FIFO queue, and q consumer threads dequeue and execute them. Each function performs simple computations (e.g., evaluating the sine of ten angles). The system is assessed by measuring the average time between task insertion and retrieval from the queue, excluding execution time. Experiments were conducted across various consumer thread counts, to identify configurations that minimize waiting time.

1 System Description

1.1 Producer Threads

Each producer thread generates and enqueues 1,000 work functions into the shared FIFO queue. A `workFunction` struct contains:

- a pointer to a function that computes the sine of ten predefined angles (in radians),
- an argument containing an array of those ten angles,
- and a timestamp marking the moment the task was pushed into the queue.

After all producer threads have completed their workload, the system inserts a number of *poison pill* tasks into the queue - equal to the number of consumer threads - to signal consumers to terminate gracefully once all regular tasks have been processed.

1.2 Consumer Threads

Each consumer thread continuously dequeues tasks from the shared queue. Upon receiving a regular task, the consumer:

- Records the task's waiting time by subtracting the enqueue timestamp from the current time.
- Stores the waiting time in a global array for post-processing.
- Executes the task's function on the provided argument.

If a poison pill is dequeued, the consumer thread terminates.

1.3 Post-Execution Analysis

Once all producer and consumer threads have completed execution, the program computes the average waiting time by aggregating values from the global array of recorded times. This metric is used to evaluate the performance of the system in different thread counts of consumers.

2 Results

The average waiting time for each producer–consumer combination was not derived from a single experiment. Instead, for every unique combination of producer and consumer thread counts, a batch of experiments was executed, and the reported waiting time represents the average across those runs. This approach was used to reduce variability and ensure statistically meaningful results.

2.1 Discussion of Results

As illustrated in Figure 1, the average task waiting time generally decreases as the number of consumer threads increases, for a fixed number of producer threads. This behavior is expected: when there are many producers and only a few consumers, the queue quickly fills with tasks. The limited number of consumers cannot process tasks as fast as they arrive, resulting in longer queue residence and thus higher average waiting times.

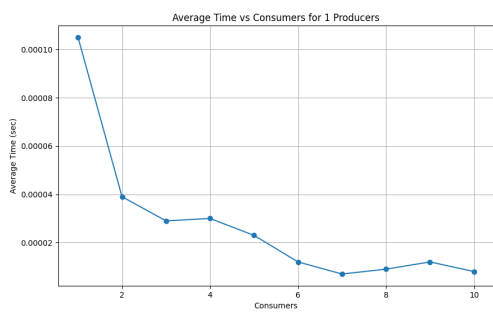
In contrast, when the number of consumers exceeds the number of producers, more consumers remain idle. In such cases, newly enqueued tasks are often consumed almost immediately by waiting consumers, significantly reducing their time spent in the queue. This leads to lower average waiting times.

However, it is important to note that excessive consumer threads may lead to diminishing returns or even minor performance penalties due to increased context switching and contention, especially on systems with a limited number of physical cores.

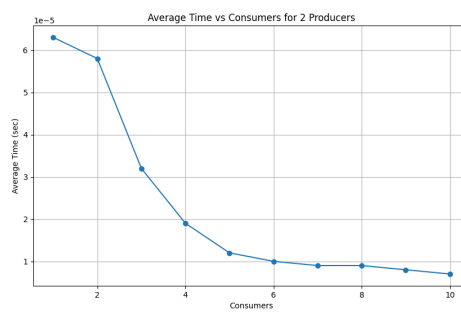
2.2 Hardware Specifications

The experiments were conducted on an Arch Linux guest system running in an Oracle VirtualBox virtual machine, with the following configuration:

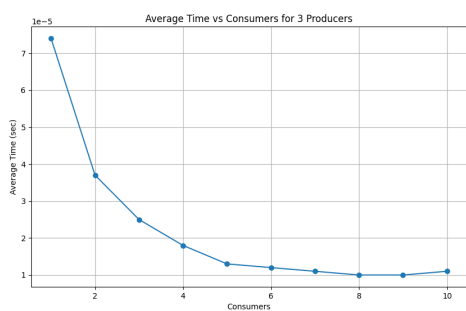
- Virtual Processor Allocation: 3 virtual CPUs
- Host Processor: 12th Gen Intel(R) Core(TM) i7-1255U @ 2.60 GHz
- Base Memory: 4 GB RAM
- Virtualization: KVM Paravirtualization with Nested Paging



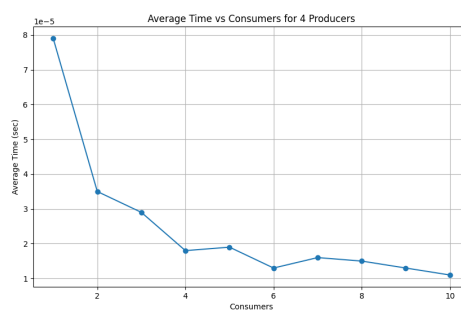
(a) 1 Producer



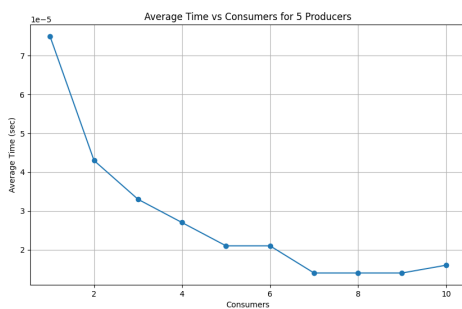
(b) 2 Producers



(c) 3 Producers



(d) 4 Producers



(e) 5 Producers

Figure 1: Average waiting time versus consumer threads.